

Bug-Tracker & Error Log Explainer Guide

A comprehensive project blueprint, technical implementation roadmap, and VS Code extension guide tailored for engineering students and internship applications.

Project Blueprint

1. The Core Idea: Bug-Tracker & Error Log Explainer

- **The Real-World Problem:** Beginners and junior developers waste hours staring at cryptic stack traces, compiler errors, or terminal logs without understanding the root cause or how to fix them.
- **Why it fits a 2nd-Year B.Tech level:** It is essentially a form submission app that takes error text and outputs a solution. It directly solves a pain point experienced during everyday coding.
- **Tech Stack Execution:**
 - **React:** A simple developer dashboard with a dark-mode code editor input box where you paste your crash log/error code and select your programming language (Python, C++, JS).
 - **FastAPI:** Endpoint that takes the error log, structures it, sends it to an LLM, and returns a clean JSON response containing: 1) What caused the bug, 2) Step-by-step fix, and 3) Corrected code snippet.
- **Future GenAI Scope:** Build a VS Code extension that automatically captures your terminal errors and sends them to your FastAPI backend with a single click.

2. Resume Rating for Freshers

Score: 7.5 out of 10

Why it's a solid project (The 7.5):

- **Solves a Real Developer Pain Point:** Interviewers love projects built for developers because they deal with messy stack traces and compilation errors every day.
- **Full-Stack Separation:** Cleanly separates concerns—React handles the UI and UX, while FastAPI handles asynchronous request routing, data validation (via Pydantic), and external API integration.
- **AI Integration without the Cliché:** Avoids generic PDF chatbots or basic ChatGPT clones by focusing on a specific utility tool for code errors.

How to turn it into an 8.5 or 9/10 for Internships:

- **Add a Database (SQLite + SQLAlchemy or PostgreSQL):** Save past errors to a history log in your backend. Add a sidebar on React showing "Past Debug Sessions".
- **Structured JSON Output:** Use Pydantic or Gemini's structured output to return clean JSON (root cause, steps, fixed code) instead of plain text, rendering them into distinct UI cards.
- **Add Markdown/Syntax Highlighting:** Install `react-markdown` and syntax highlighters so code snippets render cleanly.

3. Implementation Roadmap

Step 1: Set Up the FastAPI Backend

Install dependencies and create `main.py`:

```
pip install fastapi uvicorn google-genai pydantic python-dotenv
```

```
from fastapi import FastAPI
from pydantic import BaseModel
from google import genai
import os

app = FastAPI()
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

class ErrorRequest(BaseModel):
    language: str
    error_log: str

@app.post("/analyze-error")
async def analyze_error(data: ErrorRequest):
    prompt = f"""
    You are an expert programming assistant. Analyze this {data.language} error log
    {data.error_log}

    Provide the response in a clear, structured format:
    1. Root Cause: Explain simply why this error happened.
    2. Fix: Step-by-step instructions to resolve it.
    3. Corrected Code: Provide a brief code snippet showing the fix if applicable.
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt,
    )

    return {"analysis": response.text}
```

Step 2: Build the React Frontend

- **UI Layout:** Dropdown for language selection, textarea for stack trace/error, submit button, and a dark-mode output container.

- **API Integration:** Send POST requests using `fetch` or `axios` to `http://localhost:8000/analyze-error`.

4. Turning it into a VS Code Extension

Move your project from a web app to a developer productivity tool by turning it into a VS Code extension.

Step 1: Install Extension Tools & Scaffolding

```
npm install -g yo generator-code  
yo code
```

Select TypeScript, name your extension (e.g., `bug-fixer`), open the folder with `code .`

Step 2: Add Command Logic in `extension.ts`

```
import * as vscode from 'vscode';

export function activate(context: vscode.ExtensionContext) {
  let disposable = vscode.commands.registerCommand('bug-fixer.explainError', async () => {
    const editor = vscode.window.activeTextEditor;
    if (!editor) {
      vscode.window.showErrorMessage('Open a file and select an error log first!');
      return;
    }

    const selection = editor.selection;
    const errorText = editor.document.getText(selection);

    if (!errorText) {
      vscode.window.showErrorMessage('Please highlight an error message or stack trace!');
      return;
    }

    vscode.window.showInformationMessage('Analyzing error with AI...');

    try {
      const response = await fetch('http://localhost:8000/analyze-error', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ language: 'cpp', error_log: errorText })
      });

      const data = await response.json();
      vscode.window.showInformationMessage(`Fix: ${data.analysis}`);
    } catch (error) {
      vscode.window.showErrorMessage('Failed to connect to backend server.');
```

Step 3: Register in `package.json` & Test

Add the command under `contributes`:

```
"contributes": {  
  "commands": [  
    {  
      "command": "bug-fixer.explainError",  
      "title": "Explain Error with AI"  
    }  
  ]  
}
```

Press **F5** to launch the Extension Development Host and test your command.